

Spring Framework 6 Fundamentals

Version check info:-

-This version was created by using:-

-Java 17

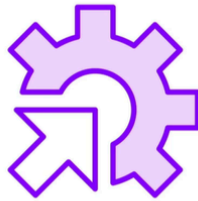
-Spring Boot 3

-Spring Framework 6

What is Spring?



Inversion of Control Container



Dependency Injection



Java without Enterprise JavaBeans (EJBs)

The technique is often referred to as dependency injection.

- 1.Spring framework just started just as an inversion of control container; the techniques often referred as dependency injection.
2. It was conceived to reduce or replace some complex configurations of the earlier Java enterprise edition.
3. Spring,without EJBs, so its original concept was how to work better with EJBs, and they just realized they didn't need those for many situations.
4. It just transitions into , how can I do the same development without using Enterprise Java beans?
5. Spring is nothing but a dependency injection container.

IOC:-

Inversion of Control (IoC) is a design principle in software development where the control of object creation and management is transferred from the programmer to the framework or container. This allows the framework to manage dependencies and lifecycle of objects, promoting loose coupling between components.

In simpler terms, instead of creating and managing dependencies manually, IoC allows an external entity (like Spring) to handle these tasks, improving flexibility and testability.

A common way to implement IoC is through **Dependency Injection (DI)**, where objects are passed their dependencies rather than creating them internally.

Dependency Injection (DI):-

Dependency Injection (DI) is a way to provide objects with the things they need to work (dependencies) instead of making them find or create those things themselves.

For example, imagine a **Car** needs an **Engine** to run. In DI, instead of the **Car** creating its own **Engine**, someone else gives it the **Engine**. This makes the **Car** more flexible because you can easily change the **Engine** it uses without changing the **Car** itself.

In simple terms, DI helps in reducing the direct dependency between objects, making them easier to change and test. The framework (like Spring) automatically takes care of injecting these dependencies for you.

Types of Dependency Injection:

1. **Constructor Injection:** Dependencies are passed to the object via its constructor.

```
1 public class Car {
2     private Engine engine;
3
4     public Car(Engine engine) {
5         this.engine = engine;
6     }
7 }
8
```

2. **Setter Injection:** Dependencies are provided through setter methods after the object is created.

```
1 public class Car {
2     private Engine engine;
3
4     public void setEngine(Engine engine) {
5         this.engine = engine;
6     }
7 }
```

3. **Interface Injection:** Dependencies are provided through an interface, which requires the object to implement certain methods to accept the dependency.

```
1 public interface EngineInjector {
2     void injectEngine(Engine engine);
3 }
4
5 public class Car implements EngineInjector {
6     private Engine engine;
7
8     @Override
9     public void injectEngine(Engine engine) {
10         this.engine = engine;
11     }
12 }
```

In **Spring**, DI is commonly used to manage objects and their relationships, with the framework handling object creation and wiring automatically through configuration files (XML) or annotations (like `@Autowired`).

What is Spring:-

It is a framework built around reducing the complexities of enterprise Java development and later providing enterprise development without EJBs. Spring can essentially be used with or without EJBs and is primarily now used without them.

 Spring enables us to develop without an application server.

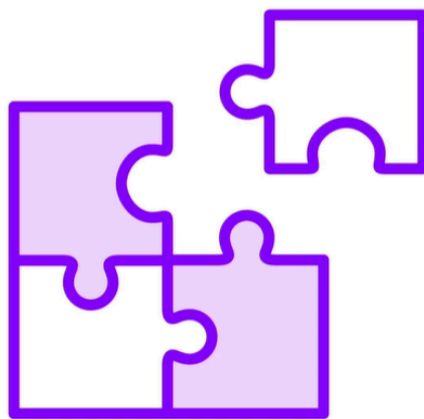
 Tomcat is not an application server; it is just a web server. This is one of the reasons that Tomcat has taken over for Java development as a standard container: It's very easy to use and lightweight. Until spring, you either weren't using enterprise features or you had some more complex, harder-to-use application server.

Lot of times now we actually use an embedded tomcat instance or jetty inside of a jar , so we don't even realize we're using tomcat for a web server because its embedded in the executable jar.

POJO Based:- Any code you write in Spring can be written without using Spring at all, POJO. is stands for plain old Java object. Spring is just helping us to write code cleaner and better and do things POJO-based and interface-driven.

What problem Spring is trying to solve:-

The Problem



Testability
Maintainability
Scalability
Complexity
Business Focus

Q: We understand it reduces the configuration, increases the scalability, and increases business focus, but how does that work?

:- 1. It is because everything in the spring is a simple POJO or plain old Java object.

2. Spring itself can be thought of as a glorified HasMap; it is a little bit more technical than that, but it's actually called the application context, and it helps us resolve our resources.

3. The application context is the configured spring container with all of the dependencies wired up in it.

Spring Configuration using Java

Java Configuration in Spring refers to using Java classes instead of XML to configure Spring beans and their dependencies. This approach is more intuitive and allows for better readability, type safety, and IDE support compared to XML configuration.

1. **No applicationContext.xml:** There is no applicationContext file; we still have the context, but it is not configured using XML.

Key Annotations in Java Configuration:

3. **@Configuration**: Marks a class as a configuration class that will define beans.
4. **@Bean**: Indicates that a method will return an object that Spring should manage as a bean.
5. **@ComponentScan**: Automatically detects and registers beans from specified packages.
6. **@Autowired**: Automatically injects dependencies into a bean

Example of Java Configuration

```
1 import org.springframework.context.annotation.Bean;
2 import org.springframework.context.annotation.Configuration;
3
4 @Configuration
5 public class AppConfig {
6
7     @Bean
8     public Car car() {
9         return new Car(engine());
10    }
11
12    @Bean
13    public Engine engine() {
14        return new Engine();
15    }
16 }
17
```

Explanation:

- **@Configuration**: This tells Spring that this class contains bean definitions. This annotation is class level annotation.
- **@Bean**: The methods annotated with `@Bean` will be executed by Spring to create and manage the returned object as a bean.
- The `car()` method returns a `Car` object, which depends on the `engine()` method to get an `Engine` object.

Using the Configuration:

```
1 import org.springframework.context.ApplicationContext;
2 import org.springframework.context.annotation.AnnotationConfigApplicationContext;
3
4 public class MainApp {
5     public static void main(String[] args) {
6         ApplicationContext context = new AnnotationConfigApplicationContext(AppConfig.class);
7
8         Car myCar = context.getBean(Car.class);
9         myCar.start(); // Using the car object with the injected engine
10    }
11 }
12
```

Benefits of Java Configuration:

- **Type Safety**: You can work with actual Java objects, catching errors at compile time.
- **Readability**: Java code is generally easier to read and maintain than large XML configurations.
- **Refactor-Friendly**: Refactoring tools in IDEs (like renaming a class) work better with Java-based configuration.

In modern Spring applications, Java-based configuration is the preferred approach due to its flexibility and simplicity.

1. Configuration Annotation:-

```
@Configuration
public class AppConfig {
    @Bean(name = "customerRepository")
    public CustomerRepository getCustomerRepository() {
        return new HibernateCustomerRepositoryImpl();
    }
}
```

@Configuration

applicationContext.xml replaced by @Configuration

@Configuration at class level

Spring Beans defined by @Bean

Setter Injection:-

Setter Injection is a type of Dependency Injection (DI) where the dependencies are provided to an object via setter methods after the object is created. In Spring, it allows you to inject dependencies by calling the setter methods of a class.

```
@Bean(name="customerService")
public CustomerService getCustomerService() {
    CustomerServiceImpl customerService = new CustomerServiceImpl();
    customerService.setCustomerRepository(getCustomerRepository());
    return customerService;
}
@Bean(name = "customerRepository")
public CustomerRepository getCustomerRepository() {
    return new HibernateCustomerRepositoryImpl();
}
```

Setter Injection

Simple as a method call

"Mystery" of injection goes away

Simply calling a setter

Explanation:-

Here `getCustomerService` returns that return a bean of type `customerService` or a bean named `CustomerService`. Notice, though, that we have a setter injection here as we're building our beans; we're going to call this `setCustomerRepository` method on our `customerService` instance. To do that, we need to have a method `getCustomerRepository` in our configuration that returns the `customerRepository` bean. This is going to wire the `CustomerRepository` inside of our `CustomerService`.

 Here Spring is doing lot of magic behind scenes when a bean registered, such as by default beans are singleton and will execute the method the first time its called.

Constructor Injection:-

```
@Bean(name="customerService")
public CustomerService getCustomerService() {
    CustomerServiceImpl customerService =
        new CustomerServiceImpl(getCustomerRepository());
    return customerService;
}
@Bean(name = "customerRepository")
public CustomerRepository getCustomerRepository() {
    return new HibernateCustomerRepositoryImpl();
}
```

Constructor Injection

Just like setter injection
Becoming the preferred way

Difference between Constructor and Setter Injection.

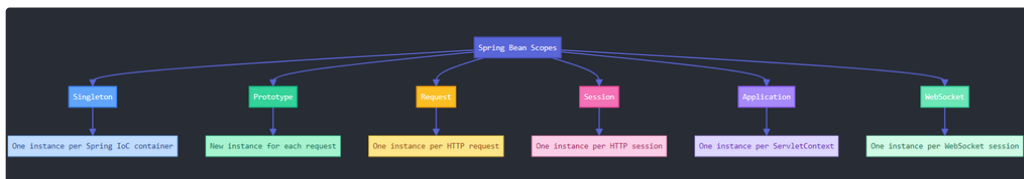
Feature/Aspect	Constructor Injection	Setter Injection
Injection Time	At the time of object creation (in the constructor).	After object creation (via setter methods).
Handling of Dependencies	Mandatory dependencies must be provided.	Allows optional dependencies.
Immutability	Promotes immutability since dependencies are set once and can't be changed.	Dependencies can be modified after creation.
Clarity of Dependencies	Clear and visible in the constructor signature.	Not immediately visible (hidden in setters).
Null Safety	Ensures all required dependencies are provided, preventing <code>NullPointerException</code> .	Risk of <code>NullPointerException</code> if not handled properly.
Flexibility	Less flexible; dependencies are fixed once provided.	More flexible; allows changing dependencies later.
Usage in Testing	Easier to inject mock objects for testing since dependencies are provided at creation.	Also allows injection of mocks but requires setter calls.
Parameter Overload	Can lead to constructors with many parameters, making code harder to read.	Avoids long constructors but can result in many setter calls.
Typical Use Case	Best for mandatory, non-changeable dependencies.	Best for optional, changeable dependencies.

Preferred Practice	Preferred for required dependencies (especially in modern Spring apps).	Useful for optional or late dependencies.
---------------------------	---	---

In summary:

- Use **Constructor Injection** when dependencies are required and should not be changed after object creation.
- Use **Setter Injection** when dependencies are optional or might need to change after the object is created.

Spring Scopes:-



In Spring, **scopes** determine the lifecycle and visibility of the beans managed by the Spring container. There are several scopes available, each defining how Spring should handle bean instances.

1. Singleton Scope (Default)

- **Description:** Only one instance of the bean is created per Spring IoC container, and the same instance is shared across the application.
- **Usage:** Default scope, typically used for stateless beans like service layers.
- **Example:**

```

1 @Scope("singleton") // or simply omit the @Scope annotation, as singleton is the default
2 @Component
3 public class MyService {
4 }

```

2. Prototype Scope

- **Description:** A new instance of the bean is created each time it is requested from the Spring container.
- **Usage:** For stateful beans or beans that require a new instance per operation.
- **Example:**

```

1 @Scope("prototype")
2 @Component
3 public class MyPrototypeService {
4 }

```

3. Request Scope (Web Applications Only)

- **Description:** A new instance of the bean is created for each HTTP request.
- **Usage:** For beans that should only exist for the duration of a single HTTP request.
- **Example:**

```

1 @Scope("request")

```

```

2 @Component
3 public class MyRequestScopedBean {
4 }

```

4. Session Scope (Web Applications Only)

- **Description:** A new instance of the bean is created for each HTTP session and is shared across requests in the same session.
- **Usage:** Useful for storing user session data.
- **Example:**

```

1 @Scope("session")
2 @Component
3 public class MySessionScopedBean {
4 }

```

5. Global Session Scope (Portlet Applications)

- **Description:** A new instance of the bean is created for the entire global HTTP session in a portlet environment.
- **Usage:** Specifically for portlet applications.
- **Example:**

```

1 @Scope("globalSession")
2 @Component
3 public class MyGlobalSessionScopedBean {
4 }

```

6. Custom Scopes

- **Description:** Spring allows you to define your own custom scopes if the default ones don't suit your needs.
- **Usage:** You can create a custom scope by implementing the `Scope` interface and registering it with the Spring container.

Choosing a Scope:

- **Singleton** is ideal for stateless, shared beans.
- **Prototype** is suitable for stateful beans where each request requires a fresh instance.
- **Request/Session** are useful in web-based applications to manage request and session lifecycles.

These scopes help manage the lifecycle of Spring beans and ensure efficient resource usage based on the application's needs.

Autowired:-

The `@Autowired` annotation in Java is a feature provided by the Spring Framework, primarily used for automatic dependency injection. Dependency injection is a design pattern that allows a class to have its required components (dependencies) supplied externally, rather than creating them internally. This promotes loose coupling and makes your code more modular and testable.

How `@Autowired` Works

When you use the `@Autowired` annotation on a field, constructor, or setter method, Spring's dependency injection mechanism will automatically inject the required bean into the class at runtime. This is done by scanning the application context for a matching bean type and injecting it where needed.

Types of Injection

1. **Field Injection:** Directly injects dependencies into fields.

```
1 public class MyService {
2     @Autowired
3     private DependencyService dependencyService;
4 }
```

2. **Setter Injection:** Uses setter methods to inject dependencies.

```
1 public class MyService {
2     private DependencyService dependencyService;
3
4     @Autowired
5     public void setDependencyService(DependencyService dependencyService) {
6         this.dependencyService = dependencyService;
7     }
8
9     // ...
10 }
11
```

3. **Constructor Injection:** Injects dependencies via the class constructor.

```
1 public class MyService {
2     private final DependencyService dependencyService;
3
4     @Autowired
5     public MyService(DependencyService dependencyService) {
6         this.dependencyService = dependencyService;
7     }
8
9     // ...
10 }
```

Constructor injection is generally recommended because it allows for immutable dependencies and makes your classes easier to test.

Qualifying Beans

If multiple beans of the same type exist, you can use the `@Qualifier` annotation alongside `@Autowired` to specify which bean to inject.

```
1 public class MyService {
2     @Autowired
3     @Qualifier("specificDependencyService")
4     private DependencyService dependencyService;
5
6     // ...
7 }
8
```

Optional Dependencies

To indicate that a dependency is optional, you can set the `required` attribute to `false`.

```
1 @Autowired(required = false)
2 private OptionalDependencyService optionalDependencyService;
```

Common Pitfalls

- **No Matching Bean Found:** If Spring cannot find a bean of the required type, it will throw a `NoSuchBeanDefinitionException`. Ensure that the bean is correctly declared and scanned.
- **Multiple Matching Beans:** If multiple beans of the same type are found and no qualifier is specified, Spring will throw a `NoUniqueBeanDefinitionException`.
- **Circular Dependencies:** Be cautious of circular dependencies, where two or more beans are dependent on each other, as this can cause runtime errors.

Best Practices

- **Use Constructor Injection:** This makes your components easier to test and promotes immutability.
- **Avoid Field Injection:** While convenient, field injection can make testing more difficult and hides dependencies.
- **Specify Qualifiers When Necessary:** Use `@Qualifier` to resolve ambiguity when multiple beans of the same type are present.

Conclusion

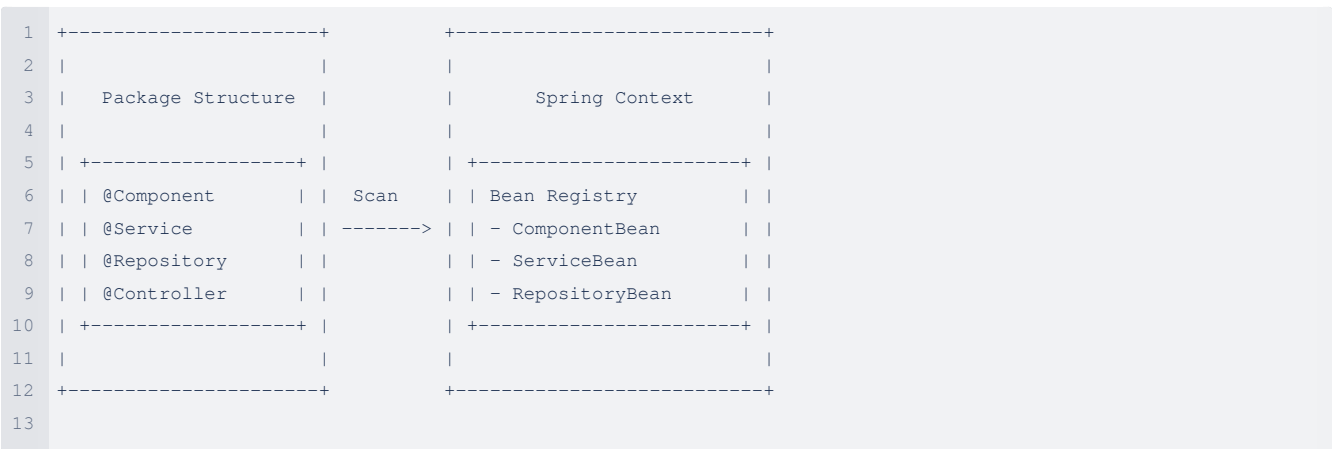
The `@Autowired` annotation is a powerful tool in the Spring Framework that simplifies dependency management in your application. By understanding how to use it effectively, you can write cleaner, more maintainable code.

References

- [Spring Framework Documentation](#)
- [Spring Dependency Injection Guide](#)

1. Spring Component Scanning Overview

Component Scanning is a mechanism in Spring that automatically detects and registers beans (i.e., components) within the application context based on specified criteria. Instead of manually declaring each bean, Spring scans specified packages to find classes annotated with specific stereotypes and registers them as beans.



Explanation:

1. **Package Structure:** Contains classes annotated with Spring stereotypes like `@Component`, `@Service`, `@Repository`, and `@Controller`.
2. **Scan:** Spring's component scanning mechanism scans the specified packages.
3. **Spring Context:** Detected classes are registered as beans within the Spring Application Context's Bean Registry.

2. Stereotype Annotations in Spring

Spring provides several **stereotype annotations** that serve as specializations of `@Component`. These annotations help in categorizing beans and can carry additional semantics, which can be leveraged for specific functionalities.

a. `@Component`

- **Purpose:** Generic stereotype for any Spring-managed component.
- **Usage:** When a class doesn't fit into more specific categories.

Example:

```
1 import org.springframework.stereotype.Component;
2
3 @Component
4 public class GenericComponent {
5     // Business logic
6 }
7
```

b. `@Service`

- **Purpose:** Indicates that an annotated class holds business logic.
- **Usage:** Typically used in the service layer.

Example:

```
1 import org.springframework.stereotype.Service;
2
3 @Service
4 public class UserService {
5     // Business logic related to users
6 }
7
```

c. `@Repository`

- **Purpose:** Indicates that an annotated class is a Data Access Object (DAO).
- **Usage:** Facilitates exception translation into Spring's `DataAccessException` hierarchy.

Example:

```
1 import org.springframework.stereotype.Repository;
2
3 @Repository
4 public class UserRepository {
5     // Data access logic
6 }
7
```

d. `@Controller`

- **Purpose:** Indicates that an annotated class serves as a web controller.
- **Usage:** Typically used in the presentation layer (e.g., handling HTTP requests).

Example:

```

1 d. @Controller
2 Purpose: Indicates that an annotated class serves as a web controller.
3 Usage: Typically used in the presentation layer (e.g., handling HTTP requests).
4 Example:

```

Diagram: Stereotype Annotations Hierarchy

```

1      +-----+
2      |  @Component  |
3      +-----+
4          ^
5          |
6      +-----+-----+-----+
7      |           |           |           |
8  +-----+ +-----+ +-----+ +-----+
9  | @Service| | @Repository| | @Controller| | Other Stereotypes|
10 +-----+ +-----+ +-----+ +-----+
11

```

Explanation:

- All stereotype annotations (`@Service` , `@Repository` , `@Controller`) are specializations of `@Component` .
- This hierarchy allows component scanning to detect any of these annotations as components.

3. How Component Scanning Works

1. Configuration:

◦ XML-Based Configuration:

```

1 <context:component-scan base-package="com.example.app"/>

```

◦ Java-Based Configuration:

```

1 import org.springframework.context.annotation.ComponentScan;
2 import org.springframework.context.annotation.Configuration;
3
4 @Configuration
5 @ComponentScan(basePackages = "com.example.app")
6 public class AppConfig {
7     // Additional bean configurations if necessary
8 }
9

```

2. Scanning Process:

Spring scans the specified base packages and sub-packages.

It identifies classes annotated with `@Component` or its specializations.

Each detected class is registered as a bean in the Application Context.

3. Bean Registration:

- Beans can be singleton or have other scopes based on their annotations or configurations.
- Dependencies are injected based on annotations like `@Autowired` , `@Inject` , or constructor injection.

Diagram: Component Scanning Flow

```

1 +-----+
2 |  Application Config  |

```

```

3 | (Java/XML Config) |
4 +-----+-----+
5 | |
6 | v
7 +-----+-----+
8 | Component Scanner |
9 | (Scans base packages) |
10 +-----+-----+
11 | |
12 | v
13 +-----+ +-----+
14 | Detected Annotations | ----> | Bean Definitions |
15 | @Component, @Service, | | (Registered in IoC |
16 | @Repository, @Controller| | Container) |
17 +-----+ +-----+
18

```

5. Practical Example with Diagrams

Let's walk through a simple Spring application structure to illustrate component scanning and stereotype annotations.

a. Project Structure

```

1 com.example.app
2 |
3 |— config
4 |   └─ AppConfig.java
5 |
6 |— controller
7 |   └─ UserController.java
8 |
9 |— service
10 |   └─ UserService.java
11 |
12 |— repository
13 |   └─ UserRepository.java
14

```

b. Configuration Class

```

1 // AppConfig.java
2 package com.example.app.config;
3
4 import org.springframework.context.annotation.ComponentScan;
5 import org.springframework.context.annotation.Configuration;
6
7 @Configuration
8 @ComponentScan(basePackages = "com.example.app")
9 public class AppConfig {
10     // Additional configurations if needed
11 }
12

```

c. Annotated Classes

```

1 // UserController.java
2 package com.example.app.controller;
3
4 import org.springframework.stereotype.Controller;
5
6 @Controller
7 public class UserController {
8     // Handle web requests
9 }
10

```

```

1 // UserService.java
2 package com.example.app.service;
3
4 import org.springframework.stereotype.Service;
5
6 @Service
7 public class UserService {
8     // Business logic
9 }
10

```

```

1 // UserRepository.java
2 package com.example.app.repository;
3
4 import org.springframework.stereotype.Repository;
5
6 @Repository
7 public class UserRepository {
8     // Data access logic
9 }
10

```

Diagram: Application Bean Registration

```

1 +-----+
2 |   AppConfig   |
3 | (ComponentScan) |
4 +-----+
5 |
6 |   v
7 +-----+
8 | Component Scanner |
9 | Scans "com.example.app"|
10 +-----+
11 |
12 |   v
13 +-----+ +-----+ +-----+
14 | UserController | | UserService | | UserRepository |
15 | (@Controller) | | (@Service)  | | (@Repository) |
16 +-----+ +-----+ +-----+
17 |               | |               | |               |
18 +-----+ +-----+ +-----+
19 |               | |               | |               |
20 |               | |               | |               |
21 +-----+ +-----+ +-----+
22 | Injected into | | Injected into | |               |
23 | Spring Context | | Spring Context | |               |

```

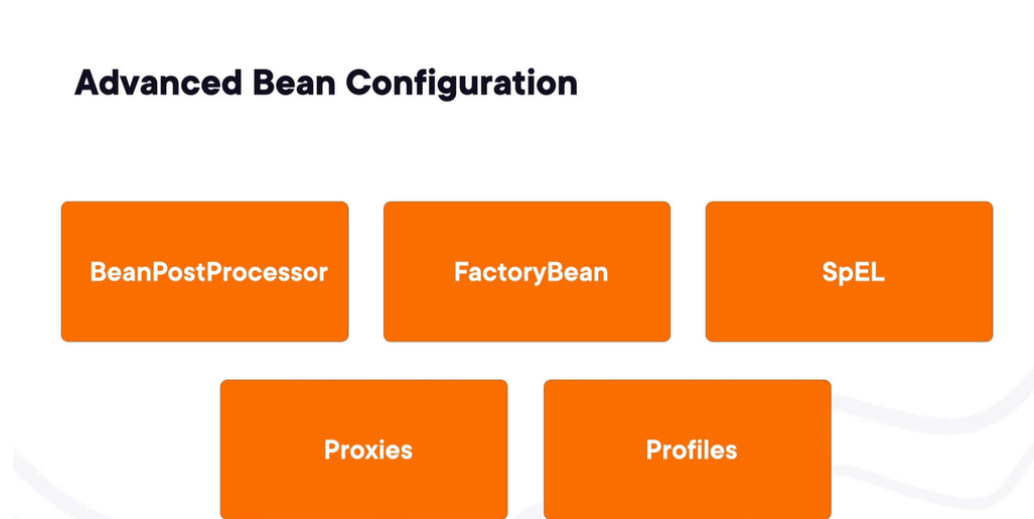
24
25

+-----+ +-----+

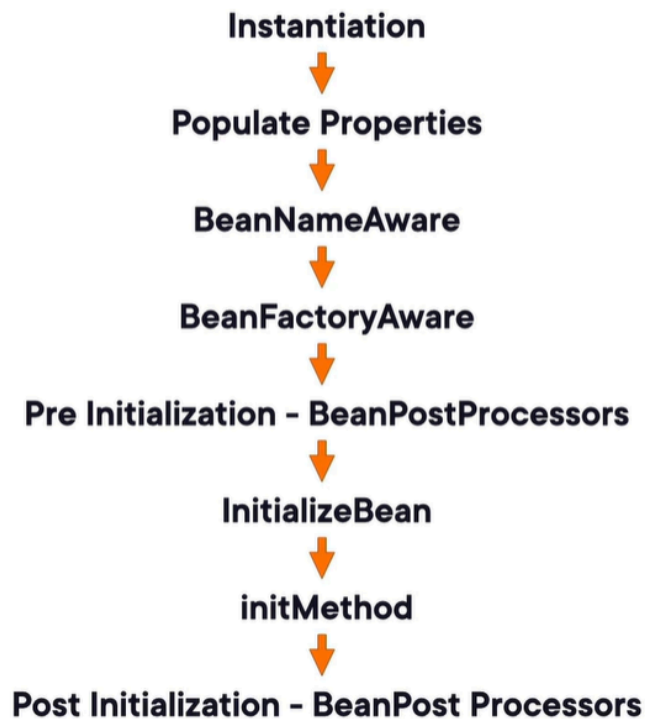
Explanation:

1. **AppConfig**: Configures component scanning for `com.example.app`.
2. **Component Scanner**: Detects `UserController`, `UserService`, and `UserRepository` based on their annotations.
3. **Bean Registration**: Each detected class is registered as a bean within the Spring Application Context.
4. **Dependency Injection**: Beans can be injected into each other as needed using `@Autowired` or other DI mechanisms.

Advanced Bean Configuration:-



Bean LifeCycle:-



@PostConstruct Annotation

- **Purpose:**

The `@PostConstruct` annotation is used to specify a method that should be executed **after** a Spring bean has been fully initialized, i.e., after dependency injection is complete.

- **When to Use:**

- To perform any initialization tasks.
- To validate that all necessary dependencies are set.
- To set up resources or configurations needed by the bean.

- **Key Points:**

- Only one method per bean should be annotated with `@PostConstruct`.
- The method must be `void`, have no arguments, and should not throw checked exceptions.
- It runs **once** during the bean lifecycle, right after dependencies are injected.

```
1
2 @Service
3 public class MyService {
4
5     @PostConstruct
6     public void init() {
7         // Initialization logic
8         System.out.println("MyService has been initialized.");
9     }
10
11     public void performAction() {
12         System.out.println("Action performed by MyService.");
13     }
14 }
15
```

FactoryBean:

In Spring, a `FactoryBean` is a special type of bean that serves as a factory for creating other beans. Unlike regular beans, a `FactoryBean` gives you more control over the instantiation process of beans, including the ability to customize the creation logic, manage complex initialization steps, or return objects that are not necessarily tied to the Spring lifecycle.

When you define a `FactoryBean`, Spring will automatically use it to create objects for you. However, the `FactoryBean` itself is not the bean; rather, it is a factory for creating the actual bean instances. You can control whether the object created by the `FactoryBean` is a singleton or a prototype.

Key Methods in FactoryBean Interface:

The `FactoryBean` interface has three main methods:

1. `T getObject()` : This method returns the instance of the bean that the `FactoryBean` creates.
2. `boolean isSingleton()` : This method determines if the bean created by the `FactoryBean` is a singleton or not. If it returns `true`, the bean is a singleton; otherwise, it's a prototype.
3. `Class<?> getObjectType()` : This method returns the type of object that the `FactoryBean` creates.

Example: Using a FactoryBean to Create Objects

Let's consider an example where we want to create a bean of `Car` using a `CarFactoryBean`.

Step 1: Define the Bean (Car)

```

1 public class Car {
2     private String model;
3     private String make;
4
5     public Car(String model, String make) {
6         this.model = model;
7         this.make = make;
8     }
9
10    // Getters and setters
11    public String getModel() {
12        return model;
13    }
14
15    public void setModel(String model) {
16        this.model = model;
17    }
18
19    public String getMake() {
20        return make;
21    }
22
23    public void setMake(String make) {
24        this.make = make;
25    }
26
27    @Override
28    public String toString() {
29        return "Car{" +
30            "model='" + model + '\'' +
31            ", make='" + make + '\'' +
32            '}';
33    }
34 }
35

```

Step 2: Implement the FactoryBean (CarFactoryBean)

```

1 import org.springframework.beans.factory.FactoryBean;
2
3 public class CarFactoryBean implements FactoryBean<Car> {
4     private String carModel;
5     private String carMake;
6
7     public void setCarModel(String carModel) {
8         this.carModel = carModel;
9     }
10
11    public void setCarMake(String carMake) {
12        this.carMake = carMake;
13    }
14
15    @Override
16    public Car getObject() throws Exception {
17        return new Car(carModel, carMake);
18    }
19
20    @Override

```

```

21     public Class<?> getObjectType() {
22         return Car.class;
23     }
24
25     @Override
26     public boolean isSingleton() {
27         return true; // or false depending on your need
28     }
29 }

```

If you're using Java-based configuration (annotations):

```

1  @Configuration
2  public class AppConfig {
3
4      @Bean
5      public CarFactoryBean carFactoryBean() {
6          CarFactoryBean factoryBean = new CarFactoryBean();
7          factoryBean.setCarModel("Model S");
8          factoryBean.setCarMake("Tesla");
9          return factoryBean;
10     }
11
12     @Bean
13     public Car car(CarFactoryBean carFactoryBean) throws Exception {
14         return carFactoryBean.getObject();
15     }
16 }
17

```

Step 4: Using the FactoryBean

```

1  public class Application {
2      public static void main(String[] args) {
3          ApplicationContext context = new AnnotationConfigApplicationContext(AppConfig.class);
4
5          Car car = context.getBean("car", Car.class);
6          System.out.println(car); // Output: Car{model='Model S', make='Tesla'}
7      }
8  }
9

```

Important Points:

1. **Direct Access to FactoryBean:** If you need to access the `FactoryBean` itself (and not the object it creates), you can prefix the bean ID with an ampersand (&). For example, to get the `CarFactoryBean`, you can call:

```

1  CarFactoryBean factory = context.getBean("&carFactory", CarFactoryBean.class);

```

2. **Singleton vs Prototype:** By overriding the `isSingleton()` method, you can control whether the object created by the `FactoryBean` is a singleton or prototype. If `isSingleton()` returns `true`, Spring will create only one instance and return it for every request for that bean. If `isSingleton()` returns `false`, a new object will be created every time the bean is requested.

Use Cases for FactoryBean:

- **When creating objects that require complex setup:** Sometimes, creating a bean involves complex initialization that's better encapsulated in a separate factory class.

- **For integrating third-party libraries:** You may need to create objects that are not directly managed by Spring, such as objects from third-party libraries.
- **Proxy objects or dynamic creation:** You can use `FactoryBean` to return proxy objects, which are common in AOP or transaction management.

`FactoryBean` provides a lot of flexibility, especially when working with third-party libraries or when bean creation logic is non-trivial.

Spring Profiles:-

Spring Profiles is a feature of the Spring Framework that allows you to define different configurations for different environments, such as development, testing, staging, and production. It helps to manage and segregate bean definitions and configurations for different environments without altering the source code.

You can mark beans or configurations with specific profiles, and Spring will activate only those beans or configurations that match the active profile.

Why Use Spring Profiles?

- **Environment-specific configurations:** For example, in development, you might want to use an embedded database, but in production, you'll use a remote database.
- **Selective Bean Registration:** You can define beans that should only be created when a particular profile is active.
- **Externalize Configurations:** Profiles allow externalized, environment-specific configurations, which makes your application more portable.

How Spring Profiles Work

You can assign profiles at the **class level** or **bean level**. You then **activate** specific profiles at runtime, either programmatically or by setting system properties or environment variables.

Example: Bean Profiles in Spring

1. Define Beans for Different Profiles

Suppose you have a `DataSource` bean that should behave differently in **development** and **production** environments.

```
1  @Configuration
2  @Profile("dev")
3  public class DevDataSource {
4
5      @Bean
6      public DataSource dataSource() {
7          System.out.println("Development DataSource bean created!");
8          // Return a DataSource for development, e.g., an H2 embedded database
9          return new EmbeddedDatabaseBuilder()
10             .setType(EmbeddedDatabaseType.H2)
11             .build();
12     }
13 }
14
```

2. Activating Profiles

You can activate a specific profile in multiple ways.

a. Programmatically (in code)

You can activate a profile programmatically when creating the application context:

```
1 public class Application {
2     public static void main(String[] args) {
3         AnnotationConfigApplicationContext context = new AnnotationConfigApplicationContext();
4         context.getEnvironment().setActiveProfiles("dev"); // Set "dev" profile
5         context.register(AppConfig.class);
6         context.refresh();
7
8         DataSource dataSource = context.getBean(DataSource.class);
9         System.out.println(dataSource);
10        context.close();
11    }
12 }
13
```

You can activate a profile by setting an environment variable. For example, when starting your Spring Boot application, you can pass the active profile as a parameter:

```
1 $ java -jar myapp.jar --spring.profiles.active=prod
```